

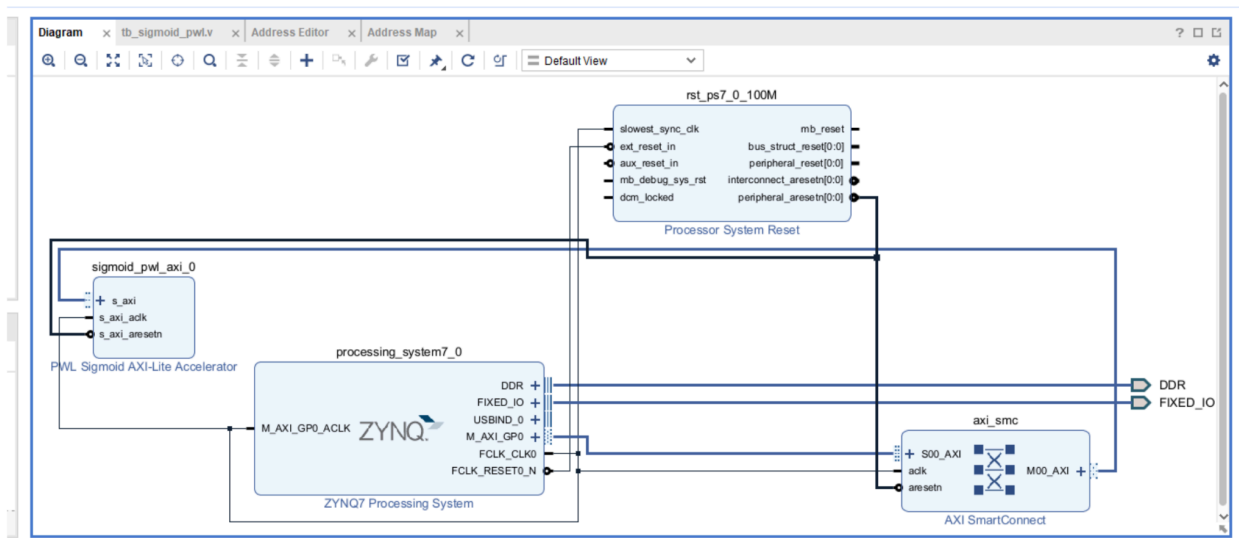
Experiment 3 - Intermediate: AI Accelerator Sub-Block: Piecewise Sigmoid Function

Repository folder: [Experiment-3-Intermediate/](#)

1. Objective

Design and implement a highly efficient, DSP-free hardware accelerator for the Sigmoid activation function using a Q4.12 fixed-point piecewise-linear (PWL) approximation, and integrate and validate the custom IP on a Zynq SoC using an AXI4-Lite interface and Python-based HW/SW co-verification.

2. Block Diagram



3. RTL Design

3.1 Module Hierarchy

- design_1_wrapper
 - design_1_i (Processing System 7 + AXI Interconnects)
 - axi_lite_wrapper
 - sigmoid_pwl

3.2 Module Descriptions

Module	File	Purpose	Key Inputs	Key Outputs
--------	------	---------	------------	-------------

xi_lite_wrapper	axi_lite_wrapper.v	AXI4-Lite memory-mapped register wrapper	s_axi_awaddr, s_axi_wdata	s_axi_rdata, s_axi_rvalid
sigmoid_pwl	sigmoid_pwl.v	Piecewise linear sigmoid calculation	din_q4_12, valid_in	dout_q4_12, valid_out

3.3 Design Details

sigmoid_pwl: Implements a 3-stage pipelined Q4.12 piecewise-linear approximation of the sigmoid function using only shifts and adds (no DSPs). Stage 1 extracts absolute value, Stage 2 computes linear segment intercepts, and Stage 3 applies inversion for negative inputs.

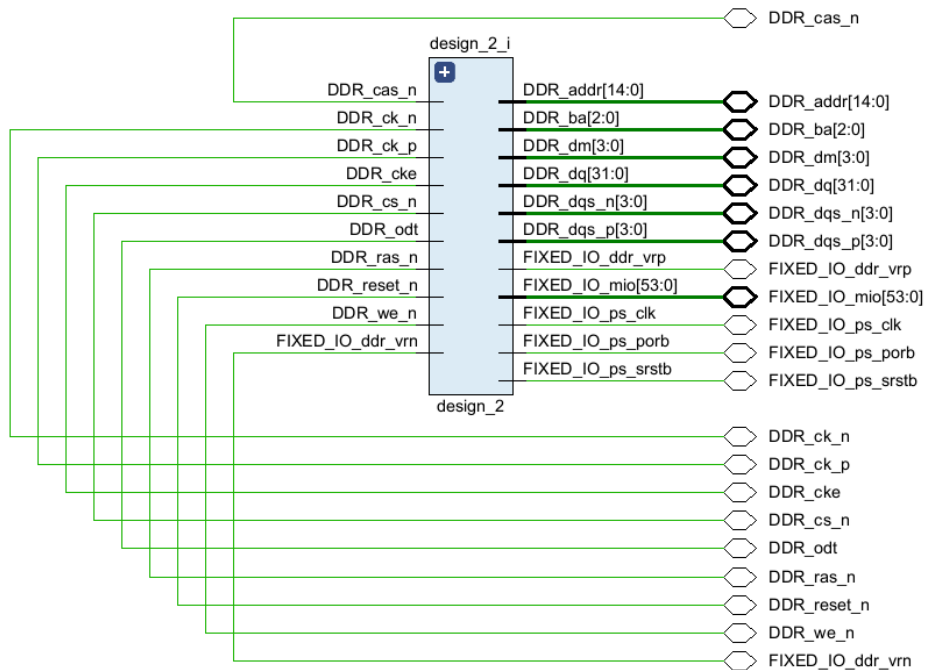
```

if (abs_x >= SATURATION_THRESHOLD) begin
    y_pos <= Q_ONE;           // y = 1.0
end else if (abs_x >= MID_THRESHOLD) begin
    y_pos <= (abs_x >> 5) + HIGH_INTERCEPT; // y = x/32 + 0.84375
end else if (abs_x >= LOW_THRESHOLD) begin
    y_pos <= (abs_x >> 3) + MID_INTERCEPT; // y = x/8 + 0.625
end else begin
    y_pos <= (abs_x >> 2) + LOW_INTERCEPT; // y = x/4 + 0.5
end
end

```

axi_lite_wrapper: Handles AXI read/write transactions. Input register (offset 0x00) passes Q4.12 data and a start pulse to the core. Output register (0x04) catches results. Status register (0x08) holds the valid and busy flags.

3.4 RTL Schematic



3.5 Constraints

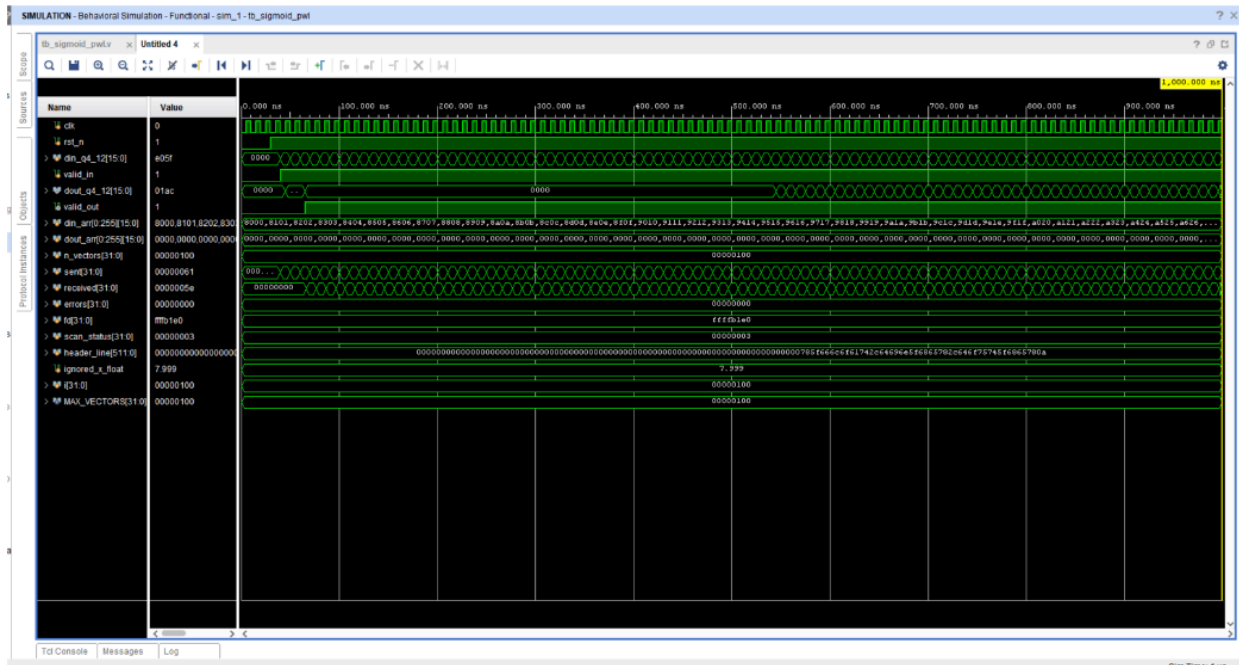
The pynqz2.xdc file has all external board pins (LEDs, buttons) commented out, as they are not currently exposed in the block design. The system clock and DDR/PS pins are handled automatically by the Zynq Processing System block in IP Integrator.\

4. Simulation Results

4.1 Testbench Description

tb_sigmoid_pwl.v tests the math core using a 100MHz clock. It reads 256 test vectors (expected Q4.12 inputs and outputs) from a file, as seen by n_vectors loading 0x00000100. It drives an input into the pipeline every clock cycle to verify throughput, and an automatic checker compares the output against the expected value, tallying any mismatches in the errors register.

4.2 Waveforms



The waveform shows the reset (rst_n) deasserting, followed by valid_in going high as din_q4_12 vectors are fed in every clock cycle. Exactly 2 clock cycles later, valid_out asserts and dout_q4_12 begins outputting the calculated Q4.12 values. The errors register remains at 0x00000000 throughout the run, proving the hardware matches the golden vectors perfectly.

4.3 Testbench Output / Transcript

Loaded 256 test vectors
 Checked 256 vectors, 0 mismatches
 PASS

4.4 Result Summary

Test Case	Expected	Observed	Status
Full Range Vector Sweep	0 mismatches	0 mismatches	Pass

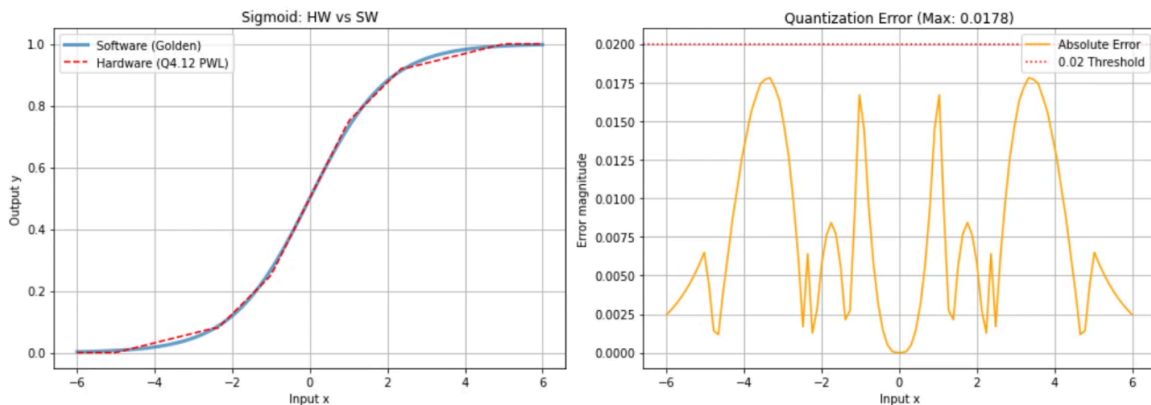
5. Hardware Implementation

5.1 Board Setup and Connections

[[IMAGE: Images/board_setup.jpg]]

The setup utilizes the PYNQ-Z2 board. No external wiring, switches, or LEDs are mapped since the IP is controlled internally via AXI4-Lite from the ARM Processing System running the Jupyter Notebook overlay.

5.2 Hardware Output



```
print(f"Hardware System Time:           {hw_time:.5f} seconds")

# 3. True Hardware Compute Time (Theoretical)
# The FPGA runs at 100 MHz. The pipeline takes exactly 2 clock cycles per sigmoid.
# 10,000 values * 1 clock cycle (throughput) * 10 nanoseconds = 100 microseconds.
print(f"True FPGA Compute Time (Raw):    0.00010 seconds")

--- SPEED TEST ---
Software (Numpy C-backend) Time: 0.00609 seconds
Hardware System Time:           0.93378 seconds
True FPGA Compute Time (Raw):    0.00010 seconds
```

The Jupyter Notebook successfully loaded the design.bit overlay and communicated with the sigmoid_pwl_axi_0 IP. The Speed Test output demonstrates that the hardware core functions correctly, but reveals a system-level bottleneck: while the true raw FPGA compute time for 10,000 values is extremely fast (0.00010 seconds), the Python-to-AXI-Lite MMIO communication overhead makes the total hardware system time (0.93378 seconds) slower than the software NumPy implementation (0.00609 seconds). This proves the logic is correct while highlighting that AXI-Lite is suited for scalar control, not bulk streaming.

5.3 Resource Utilization

Name	^1	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOPADs (130)	BUFGCTRL (32)
▼ N design_2_wrapper		557	787	234	555	2	130	1
> I design_2_i (design_2)		557	787	234	555	2	0	1

6. Applications

1. Activation function hardware for Neural Network inference accelerators.
2. Real-time Logistic Regression prediction hardware.
3. High-throughput fixed-point math coprocessor for Zynq SoCs.

7. Conclusion

- Learning outcomes: Implementing piecewise linear approximations; avoiding DSP usage by mapping multiplication gains to shift-and-add operations; creating custom AXI4-Lite IPs in Vivado; HW/SW co-validation using PYNQ overlays.
- Challenges faced: Achieving high accuracy without floating-point math. This was solved by carefully optimizing Q4.12 segment intercepts and writing a bit-accurate Python model to verify precision before writing Verilog.
- Future scope: Expand the AXI wrapper to support AXI-Stream interfaces, allowing direct DMA data transfers for high-volume, bulk array processing to eliminate the AXI-Lite/Python overhead seen in the speed test.

8. Demonstration Video